Sketchpad: a small vision-language model that computes exactly on its own sheet

Mostafa 'Neo' Mohsenvand
MIT Media Lab

31 October 2026

Abstract

Language models still get long arithmetic wrong, and the standard remedy hands the computation to a tool outside the model, so nothing the model computes itself comes with a guarantee. Here we present Sketchpad, in which a vision-language model of 0.8 billion parameters computes exactly on a rendered sheet that is its only working memory. Each forward pass of Qwen3.5-0.8B, with one low-rank adapter, reads three cells of the sheet through the model's own vision encoder and writes one step: a digit, a carry or borrow, and the name of the next step. A fixed runtime that does no arithmetic paints the answer and moves the window. Because a step sees only its view, exactness on the finite set of views is equivalent to exactness on every run of every length. We state this, its necessity, composition and transfer across hardware as theorems checked in Lean, and we certify all 2,446 views on the device that runs the model, a laptop GPU in a web browser. The certified model adds, subtracts, multiplies and divides integers of any length, takes square roots digit by digit and finds greatest common divisors: it solved 200 of 200 registered problems exactly in the browser, and 220 of 220 sums of up to 1,000 digits and products of up to 20-digit numbers, of which Qwen3.5-27B with chain of thought solved 121. On this arithmetic the runtime does algebra as a textbook writes it, with every number computed by the model and every answer checked by substitution. One graph input switches the adapter off, and the model is then stock Qwen bit for bit. Computation done by a neural network can carry a proof of its correctness that is checked where it runs.

1 Introduction

Exact computation over many steps is where language models still fail. Multiplying two long numbers, adding a thousand-digit sum or carrying a remainder through a long division takes hundreds of steps, and one wrong step spoils the answer. Chain of thought, in which the model writes out its intermediate steps before answering[1,2], extends how far a model gets, and a great deal of work has gone into making transformers generalise arithmetic to longer inputs than they were trained on[3–9]. Each of these works measures exactness on samples: none can say that the next, longer input will be right. On the 220 problems of our registered baseline, Qwen3.5-27B with chain of thought, a model with 27 billion parameters, answered 8 of 20 hundred-digit additions and none of 20 thousand-digit ones exactly within 32,768 tokens of reasoning (Fig. 1c; Methods).

The usual way out is to leave the arithmetic to a tool. Toolformer, PAL, ToRA and their successors train a model to write a call or a program and let an interpreter return the result[10–14]. That is accurate and practical, but the computation happens outside the network, and what the network itself computes stays unverified.

We take the other path and keep the computation inside the model, in a form that can be checked completely. A person doing long multiplication does not hold the numbers in their head; they write them down and look at a few digits at a time. Sketchpad gives a vision-language model, a language model that also reads images through a vision encoder, a sheet to work on. A fixed runtime draws the problem on the sheet. At each step it cuts out a strip of three cells, the *view*, and shows it to the model with a one-word prompt naming the step. In one forward pass, one evaluation of the network on its input, the model writes that step's answers: a digit, a carry or a borrow, and the name of the step to take next. The runtime paints the answers onto the sheet and moves the window where the model's answer says. The runtime holds no arithmetic, and nothing passes from one step to the next except what is painted on the sheet (Fig. 1a,b).

This design makes the model's behaviour checkable. Every step is a function of its view, and the views come from a finite set: 242 for column addition, 1,553 for the steps of long multiplication, 651 for subtraction and division, 2,446 in all. We run the model on every one of them and require the correct answer to win by

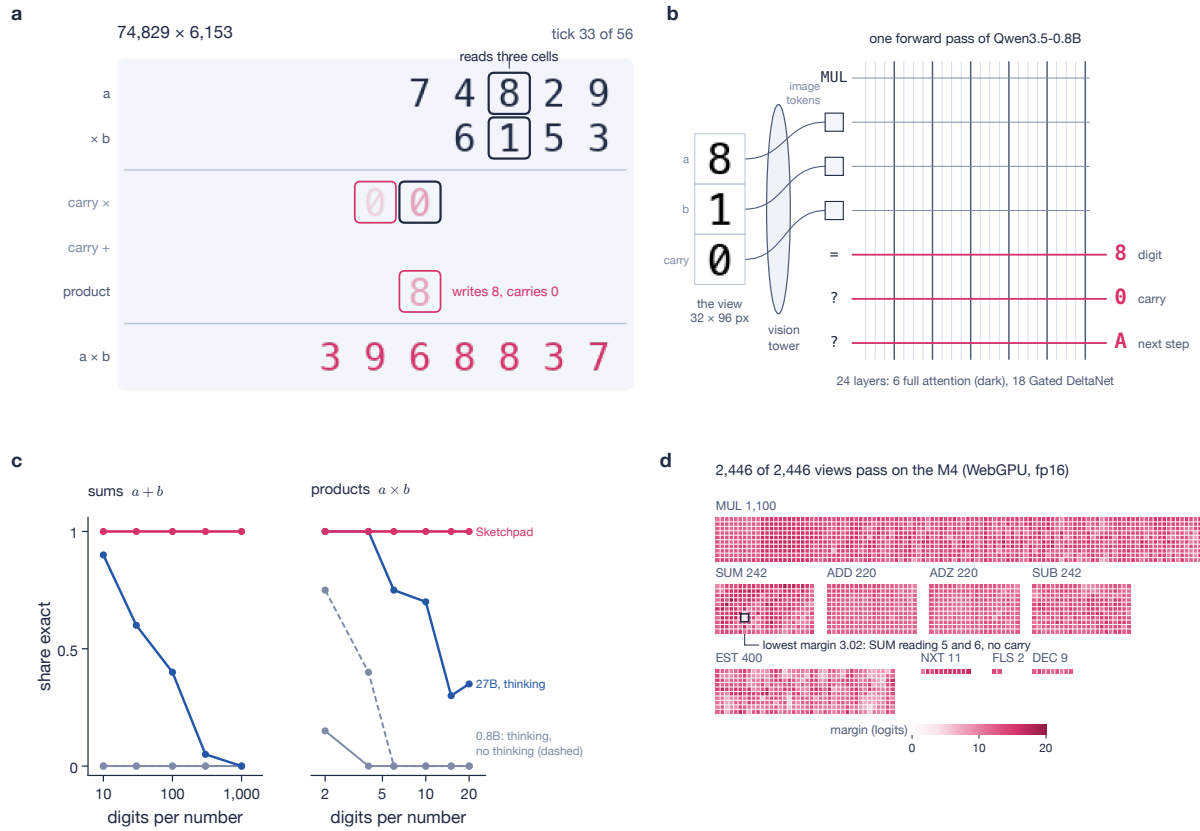


Fig. 1 | The sheet: one step, any length, one certificate. a, Long multiplication of 74,829 by 6,153 on the sheet, as the runtime lays it out, at step 33 of 56. The window (dark outline) shows the model three cells: a digit of each factor and the carry. What the model has written is in rose. **b**, One step is one forward pass of Qwen3.5-0.8B. The view, 32 × 96 pixels, enters the vision encoder as three image tokens after the step's word (`MUL`); the answers are read at `=` and at two placeholder slots: the digit, the carry and the next step (`A`, add the partial product). The 24 layers are 6 full-attention layers and 18 Gated DeltaNet layers. **c**, Share of problems answered exactly against digits per number: sums of 10 to 1,000 digits and products of 2- to 20-digit numbers, 20 problems per point (registered run, Methods). Sketchpad (rose) is exact on all 220; Qwen3.5-27B with chain of thought (cobalt) and Qwen3.5-0.8B with and without it (blue-grey) fall away with length. The certificate covers every length (dotted). **d**, The certificate: all 2,446 views, one square each, shaded by margin, as measured on the M4 in the browser. The smallest margin, 3.02 logits, is an addition view reading 5 and 6 with no carry.

a margin. Exactness on every view is then equivalent to exactness on every run of every length (Theorem 1). The idea is not new: Moore's analysis of experiments on sequential machines[15], Cai, Shin and Song's verification of learned recursive programs[16] and Lewandowski, Machado and Schuurmans' certificate for universal computation by a language model's decoding[17] rest on it. We state it for the sheet, together with its necessity, the composition of certified procedures and the transfer of a certificate between devices, and we check all four in the Lean proof assistant.

We make five contributions.

1. **The sheet machine.** Qwen3.5-0.8B[18] with a low-rank adapter (LoRA[19]) of 10.2 million parameters, 1.2% of the model, writes one step per forward pass (Section 2).
2. **A certificate on the device.** All 2,446 views pass on an Apple M4 laptop GPU in the browser, the weakest by 3.02 logits, and every visitor's GPU checks the weakest tenth again (Section 3).
3. **A calculator.** Five tables give signed sums and differences, long multiplication and division, square roots and greatest common divisors at any length; 200 of 200 registered problems were exact in the browser (Section 4).
4. **Algebra as written.** The runtime solves, expands and factors as a textbook does, with every number computed by the model and every answer checked (Section 5).
5. **A gate.** One graph input switches the adapter off, and the model is then stock Qwen3.5-0.8B bit for bit (Section 6).

2 The sheet machine

The model. Qwen3.5-0.8B is an open vision-language model with a vision encoder and a 24-layer language model[18]. Six of its layers are full attention; the other eighteen are Gated DeltaNet layers, a form of linear attention that keeps a fixed-size running state instead of a cache of every past token[20]. We add a LoRA adapter, a pair of small matrices beside each of 150 weight matrices of the language layers, whose product is added to the frozen weight's output[19]. The vision encoder and the base weights stay frozen.

Views and steps. The sheet is a grid of 32-pixel cells holding blank or a digit, drawn from one fixed set of glyph images. A view is a column of three cells, 32 × 96 pixels, which the vision encoder turns into three image tokens. A step is the prompt `WORD<image>=??`: a word naming the step (`SUM`, `MUL`, `ADD`, `SUB`, `EST`, `DEC` and four control words), the view, an equals sign and two placeholder slots. The model's answers are read at the equals sign and at the two slots of a single forward pass, so a step costs one pass however many answers it has (Fig. 1b). Because every layer of the model is causal, what follows the equals sign cannot change the answer read there, so every prompt is padded to the same length and steps of different kinds share one batch (Methods).

Tables. For each step word, the *table* lists every view the step can meet with its correct answers. Column addition (`SUM`) reads a carry and two digits and writes their sum as one token, or `H` to halt: 242 views. Long multiplication uses five steps (multiply a digit pair with the carry, add into the running total, end a pass, flush the last carry, move to the next digit): 1,553 views. Subtraction (`SUB`, 242), guessing a quotient digit (`EST`, 400) and lowering a guess (`DEC`, 9) complete the calculator: 2,446 views in nine tables (Extended Data Table 1).

The runtime. The runtime is a small TypeScript program. For each procedure it lays the numbers out on the sheet, cuts the views, sends them to the model, writes the glyphs the model returns and moves the window as the model's next-step answer says. In multiplication it holds no arithmetic and no loop: the model's own next-step token decides when a row ends and when the product is done. In division and square roots the runtime chooses the procedure's skeleton (bring down the next digit, guess, multiply and subtract), and the branches inside it are the model's: the borrow the model writes out of the top column says whether a guess was too high. Every view the runtime cuts is checked against the tables before it is sent, and a view outside them would be reported; none occurred in any run reported here.

Waves. Independent procedures can share forward passes: when an algebra problem needs many separate products, the runtime steps them in lockstep and sends one view from each in a single batch. On the M4, a pass costs about 0.065 s plus 0.037 s per view in the batch, so batching cuts the fixed cost of a step, not the work per view (Fig. 2c).

3 Certification

Margins. The model's answer at a position is the token with the largest logit, the score the network gives each possible token. A view *passes* when, at every answer position, the correct token's logit exceeds every other; the smallest difference is its *margin*, and a margin of 3 makes the correct token about 20 times more probable than any other. A *certificate* records that every view of every table passes, with the margins.

We certify twice. In PyTorch on the training server, in 32-bit floating point, the smallest margin over the 2,446 views is 3.96 (an addition view; Extended Data Table 1). In the browser, the page runs the deployed graph through ONNX Runtime Web on WebGPU[21–23] in 16-bit floating point, in exactly the padded batch form that runs; on the M4 all 2,446 views pass, with smallest margins 3.02 for addition, 6.52 for the multiplier's steps and 4.61 for the calculator's (Fig. 3c). The full check takes 3.5 minutes there. The certificate travels with the model files, fingerprinted by the hash of every file, and a visitor's GPU checks the weakest tenth of every table again, the rows with the smallest margins, before it trusts the shipped table; it can check all of them on request.

What the certificate implies. Write C for the configurations of a procedure (the whole sheet, the window's place and the procedure's control state), V for the views, and A for the answers. At a configuration c the runtime either halts or shows the model a view $v = \mathrm{view}(c)$, then paints the model's answer: $c' = \mathrm{paint}(c, f(v))$. Under the *pinned-sheet policy*, each forward pass sees the step's prompt and its view and nothing else, so the model's answers are a function $f : V \to A$. The reference table is $T : V \to A$.

Theorem 1 (Theorem P, sheet form). *Suppose the runtime paints the answer, so that different answers to a view give different sheets. Then for every start in a set I and every number of steps, the model's run equals the reference run configuration by configuration if and only if $f(v) = T(v)$ for every view v*



Fig. 2 | The sheet machine. **a**, The five step tables of the calculator, each with one real view (the model's own input pixels) and its answers in rose: `SUM` writes 6 with carry 1 from $1 + 7 + 8$; `MUL` writes 6 and carry 4 from $7 \times 6 + 4$; `SUB` writes 4 and borrow 1 from $3 - 8 - 1$; `EST` guesses 7 from $\lfloor 47/6 \rfloor$; `DEC` lowers a guess of 7 to 6. **b**, A procedure is a walk over tables. Left, the division 4,100 ÷ 59 on its sheet, where the borrow out of the top column shows that the guess 8 is too high. Right, every forward pass of that run by Knuth's Algorithm D, and the 56 passes of the square root of 2,000,000. **c**, Waves: four independent jobs share 32 forward passes instead of 73 one after another. Measured seconds on the M4 against the cost model (0.065 s per pass plus 0.037 s per view) for the 200 registered problems; the median ratio is 1.01. **d**, The 200 registered problems in the browser, 50 per set, all exact; seconds per problem against the 12-second line.

that some reference run from I meets. In particular, if every such view is a row of the tables and the model passes every row, every run of every length is exact.

LEAN: `Sketchpad.sheet_exact_iff`, `Sketchpad.sheet_certified`, `Polyformer.Pinned.regeneration_trace_iff`, `Polyformer.Pinned.pinned_regeneration_iff`

Proof sketch. Sufficiency is induction on steps: if the two runs agree up to step t, they show the model the same view, which is a row the model passes, so they agree at step $t + 1$. For necessity, take the first view where the model and the table disagree; the reference run meets it, and the two runs paint different answers there. The Lean proof instantiates the package's general regeneration theorem with the configuration as the state, so the sheet form and the token form of Theorem P are the same theorem.

Theorem 1 turns a statement about infinitely many runs into 2,446 checks, as Cai et al. did for learned recursive programs[16] and Lewandowski et al. for a language model's decoding of a universal machine, by checking its 1,857 production rules[17]. Its force lies in the hypothesis that the model's answer is a function of the view, which the pinned-sheet policy guarantees. If the model kept earlier steps in its context, as chain of thought does, no finite set of checks would suffice.

Theorem 2 (No finite test once state leaves the sheet). *Suppose each step's context keeps the earlier steps. Then for any finite set of test contexts there is a decoder, for a two-state table, that passes every row of the table and is right on every test context, all with strictly positive margins, and still makes an error on some longer run. The pinned run of the same decoder is exact at every length.*

LEAN: `Polyformer.Pinned.no_finite_test_suite`, `Polyformer.Pinned.no_finite_check`

Proof sketch. The counterexample is a correct lookup plus one attention head whose write grows with the number of a given token in the context, scaled so that it crosses the margin only after more steps than any test contains. This is Moore's observation that no finite experiment identifies a machine whose number of states is unbounded[15], realised by a decoder that passes the per-row certificate.

Long procedures are built from many certified calls: a square root runs multiply-subtract passes, an algebra problem runs dozens of products and quotients whose inputs quote earlier results.

Theorem 3 (Composition). *Let a transcript make K calls in order, each a certified procedure on its own sheet, whose inputs may quote the results of earlier calls. Every call's result equals its table's output on the input it actually received, at any lengths and in any order. If the final answer is wrong, then some table is wrong, some quote is wrong, or the glue that combines the results is wrong; for any distribution of transcripts, the probability of a wrong answer is at most the sum of the probabilities of those three events.*

LEAN: `Polyformer.Pinned.compose_exact`, `Polyformer.Pinned.error_decomposition_certified`, `Polyformer.Pinned.error_bound`

For the calculator the tables are fixed and correct by construction, so a wrong answer can come only from a wrong transcription of the problem or wrong glue. In the algebra of Section 5, the glue is the runtime's textbook procedure and every answer it returns is checked, so a wrong transcription is the remaining risk.

A certificate is measured on one code path: one device, one numerical precision, one batch shape. The run may take another.

Theorem 4 (Transfer across code paths). *Suppose that at every view a run can meet, the logits on the path that runs differ from the certified logits by at most τ at every output. If every certified row has margin greater than 2τ, the run is exact on every input of every length.*

LEAN: `Polyformer.Pinned.sel_eq_of_close`, `Polyformer.Pinned.regeneration_of_close`, `Polyformer.Pinned.policy_tick_eq`

Proof sketch. At each view the correct token leads by more than 2τ on the certified path; moving every logit by at most τ cannot overturn that lead, so each step picks the same answer, and Theorem 1 applies. The package also proves that a pinned step computed through a cached prompt prefix, with the step's tokens evicted after each step, gives the same logits as a fresh forward pass, so that choice of cache is another path with $\tau = 0$ in exact arithmetic.

Theorem 4 explains what we measured on hardware. A 4-bit quantised decoder moved some add rows of an earlier adapter from a margin of +9.4 on the server to −8.9 on the M4, far more than any margin, so the page ships the 16-bit decoder, which reproduces the server's certificate. Padding the addition prompt to the batch form moved no margin by more than 0.0018. The device certificate is measured on the path that runs,

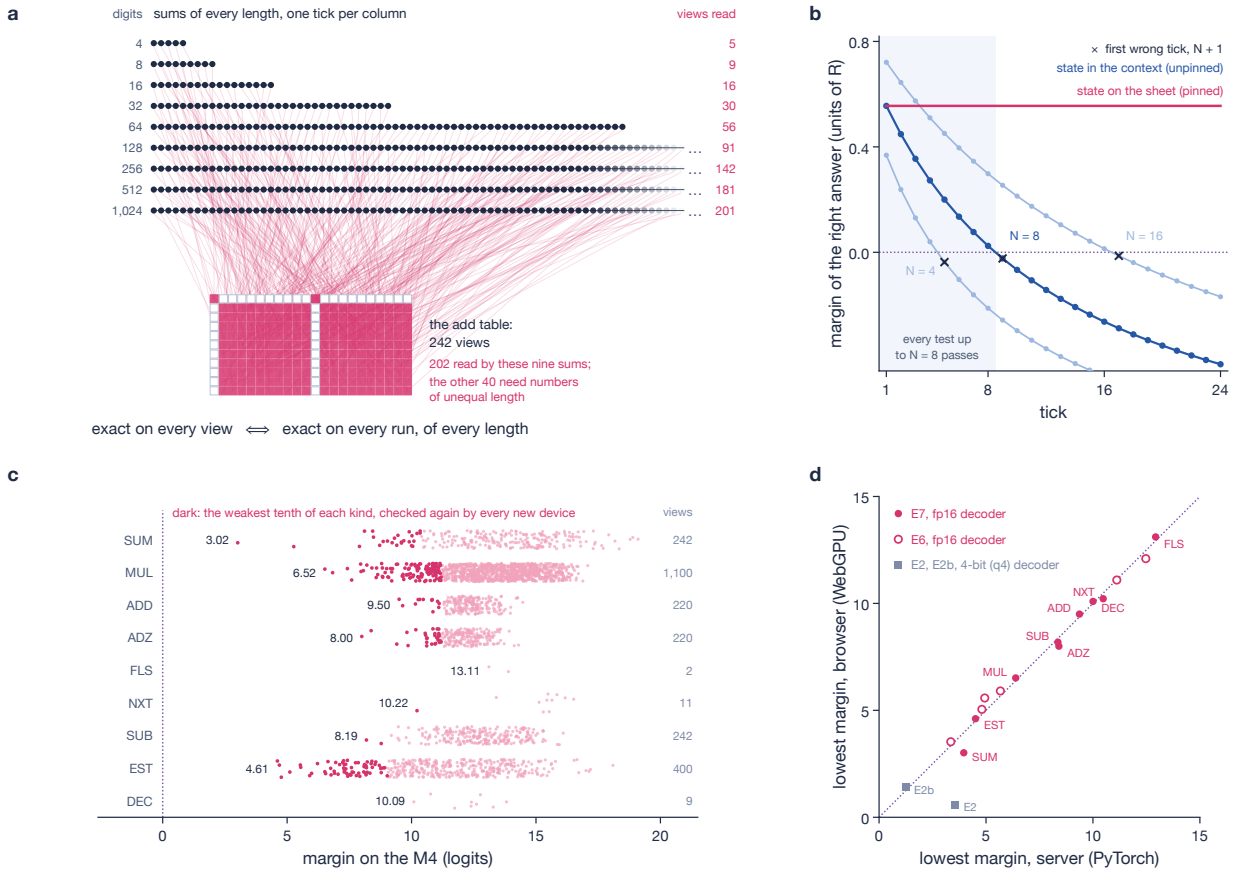


Fig. 3 | Certification. a, Sums of every length, one step per column, read only views from the 242-view addition table; nine sums from 4 to 1,024 digits already read 202 of them, and the other 40 need numbers of unequal length. Exact on every view is exact on every run (Theorem 1). **b**, The counterexample of Theorem 2: when the state lives in the context, a decoder can pass every test up to N steps and fail at step $N + 1$ (blue, for $N = 4$, 8 and 16), while the pinned run keeps its margin at every step (rose). **c**, Margins of all 2,446 views on the M4, per step word; the weakest tenth of each, which every new device checks again, is dark. **d**, Smallest margin per step word on the server (PyTorch, 32-bit) against the browser (WebGPU): the 16-bit decoder follows the identity line for the published model (E7) and the one before it (E6); the 4-bit decoder of the first adders (E2, E2b) moved the smallest margin by up to 3 logits.

so there τ is only the variation between runs; two certificates taken on the M4, with and without the gate, agree on all 2,446 margins to the stored thousandth. On another GPU, the spot check of the weakest tenth measures how far the margins have moved (Fig. 3d).

4 A calculator on the sheet

Every key of a pocket calculator is a pen-and-paper procedure over a few digit tables. Signed sums and differences use addition or subtraction column by column; the borrow out of the last column says which number was larger, so the sign comes free. Multiplication is the schoolbook method, one multiply step per digit pair and one add step per partial-product digit.

Division follows Knuth's Algorithm D, the standard method for dividing long numbers[24]. Both numbers are first scaled by a single digit so that the divisor's leading digit is at least 5, which costs one multiplication pass. Each quotient digit is then guessed from the top two digits of the remainder and the top digit of the divisor, the `EST` table. After scaling, the guess is never too small and at most two too large. The runtime multiplies the divisor by the guess and subtracts it from the remainder in one pass; if the model's borrow out of the top column is 1, the guess was too high, `DEC` lowers it and the divisor is added back.

Square roots are taken digit by digit, the method taught in schools, bringing the number down two digits at a time. We scale the number by c^2, with c one of 1, 2, 4 or 5, so that the root starts with 25 to 49; twice the root then starts with 5 to 9, so each digit's guess is division's guess, at most two too high, and the same

three tables serve. The **greatest common divisor** is Euclid's algorithm by repeated division.

We registered the test before running it: 50 problems each of division to 10 decimal places, square roots to 6 places, greatest common divisors of numbers up to 999, and signed sums and differences of up to 20 digits, run one after another in the browser on the M4. All 200 answers were exact, and every view of every run was a certified row. Median times were 7.0 s for division, 9.8 s for square roots, 7.0 s for the greatest common divisor and 2.0 s for sums (Fig. 2d; Extended Data Table 2). 184 of 200 finished within 12 s; the slowest were roots of three-digit numbers, at up to 16.5 s, because the trial divisor grows a digit with every root digit and a guess that is too high costs a whole pass. Measured times followed the cost model with a median ratio of 1.01.

On a server GPU, where many problems can run in one batch, the same model answered all 220 problems of the registered baseline, sums of 10 to 1,000 digits and products of 2- to 20-digit numbers: 64,700 views, all certified rows, in 1,001 batched forward passes. Qwen3.5-27B with chain of thought, run with the same prompts and a 32,768-token budget, answered 121 of the 220, falling from 18 of 20 ten-digit sums to none of the thousand-digit ones and from 20 of 20 four-digit products to 7 of 20 twenty-digit ones; 93 of its 99 failures reached the token budget. Stock Qwen3.5-0.8B answered 3 of 220 with chain of thought, nearly always looping until the budget ran out, and 23 without (Fig. 1c; Extended Data Table 5).

5 Algebra as written

Algebra in school is arithmetic organised by a few rewriting rules. The runtime writes a derivation the way a textbook would, one line per step, and every number on every line comes from a certified procedure on the sheet. To solve $3(x-2)+5=2x+14$, it multiplies out the bracket, which needs the product $3\cdot 2$; collects $-6+5$; moves the terms across; divides; and substitutes $x=15$ back into both sides, which gives 44 on each. Quadratics use the discriminant and the quadratic formula, with a square root that ends with remainder zero exactly when the roots are rational. Powers of a binomial are expanded by the binomial theorem, with Pascal's rule supplying the coefficients by addition. Polynomials are factored by the rational root theorem, which says that a rational root of an integer polynomial is a divisor of the constant term over a divisor of the leading coefficient, and by synthetic division, the shortcut for dividing by $x-r$; systems of equations by elimination (Fig. 4a). Irrational roots of quadratics are given exactly, as simplified radicals such as $3\pm\sqrt{5}$, checked through Vieta's formulas, that the roots sum to $-b/a$ and multiply to c/a, in integer arithmetic.

Every answer is checked. Rational roots are substituted back into the equation, expansions are compared with the original at a further point, and irrational root pairs pass the Vieta identities exactly; an answer that fails is refused and the failed check is shown. A substitution check confirms the roots found, not that all roots were found; the runtime knows the degree and reports when a factor remains unresolved. With deliberately planted faults, the checks refused 200 of 200 wrong roots offered as answers, and 185 of 200 runs in which one call's arithmetic was corrupted; in the 15 runs they did not refuse, the corrupted number did not change the answer (Fig. 4b). The certificate already covers the arithmetic; the checks guard what it does not cover, the runtime's algebra and the transcription of the problem, and as the planted faults show they also catch most arithmetic faults.

Cost. The algebra is classical and so is the comparison. Computer algebra can also solve by evaluating both sides at a few numbers and reading the polynomial off the values (interpolation[25]), or by evaluating at a power of ten so the coefficients appear in groups of digits (Kronecker substitution[26]: $(x+2)^5$ at $x=1000$ is $1\,010\,040\,080\,080\,032$). That route needs only arithmetic too, but it hides the algebra and costs more rows on the sheet. By the cost model, which the calculator's measured times follow (median ratio 1.01), the textbook route takes 2.8 s for the linear equation above against 4.5 s by evaluation, 12.1 s for $(x+2)^5$ against 33.1 s, and 16.9 s to factor $x^3-6x^2+11x-6$ against 31.5 s. Six of the eleven worked examples take less than 12 s by the cost model, by either route and either check. Measured on the M4 with the default check, eight do: the expansions of $(x+2)^5$ and $(2x-3)^4$ finish in 9.5 and 10.4 s, while $(x+1)^{10}$, cubic factoring and a three-by-three system take 16 to 22 s. The exact irrational roots of three quadratics take 7.4 to 11.2 s, and their four-place decimal approximations, which the page shows after the exact answer, about as long again (Fig. 4c; Extended Data Table 3).

6 One model that decides

Teaching a model a new skill tends to erode its old ones, the problem of continual learning[27,28]. Our arithmetic adapters barely move the model's perplexity on plain text, the usual check: the multiplier adapter E6 raised

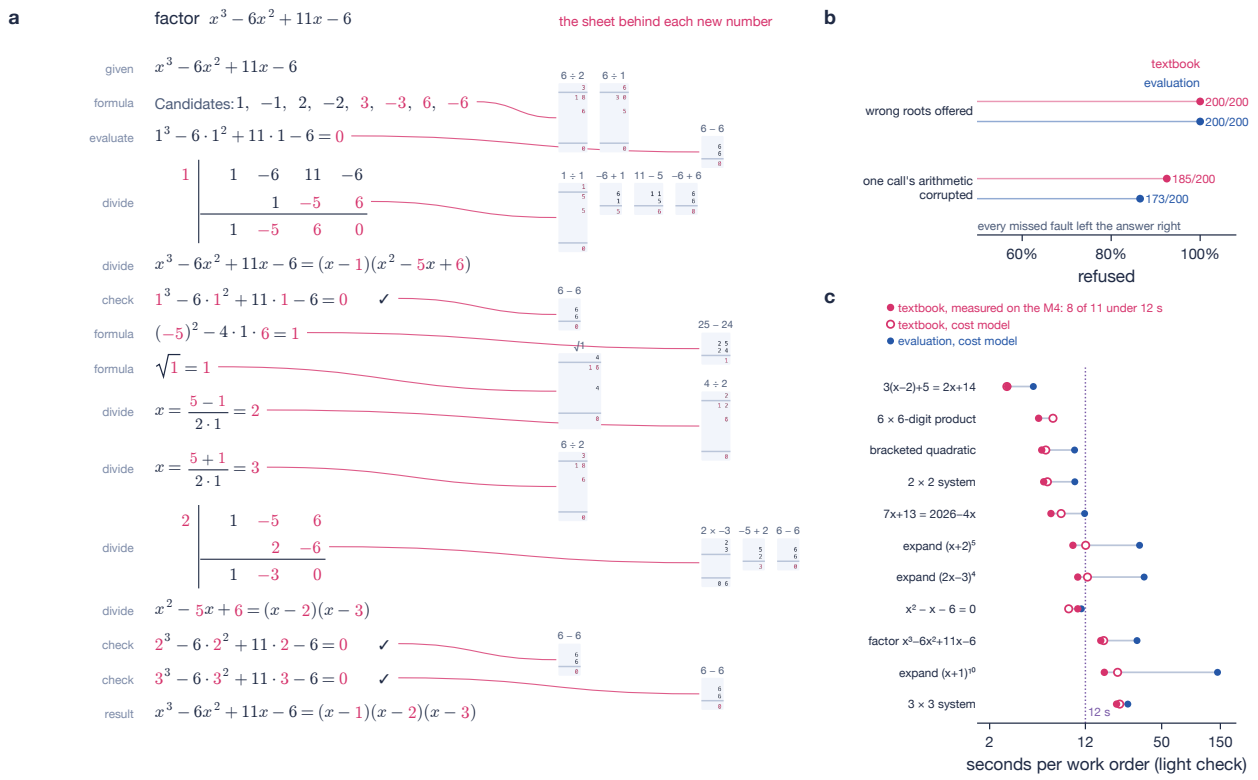


Fig. 4 | Algebra as written. **a**, Factoring $x^3 - 6x^2 + 11x - 6$, typeset as the runtime writes it. Each number in rose was computed by the model on its sheet; the thread leads from the number to the small column sum or product that produced it. Numbers copied from the problem stay in ink. **b**, Refusals under planted faults: wrong roots offered as answers (200 of 200 refused by both routes) and one call's arithmetic corrupted (185 of 200 refused by the textbook route, 173 of 200 by evaluation); every fault that was not refused left the answer right. **c**, Seconds for eleven worked examples with the default check: the textbook route measured on the M4 (rose) and by the cost model (open rose), against evaluation and interpolation by the cost model (cobalt), with the 12-second line.

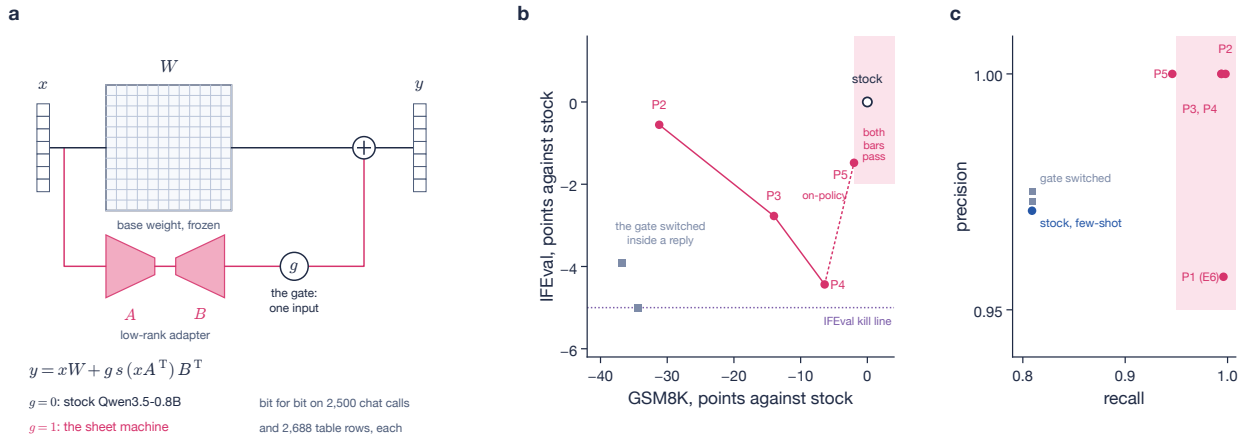


Fig. 5 | One model that decides. **a**, The gate: one input of the graph multiplies every adapter branch. At $g = 0$ the model is stock Qwen3.5-0.8B bit for bit, on 2,500 chat forward passes and 2,688 table views; at $g = 1$ it is the sheet machine. **b**, Instruction following (IFEval) against multi-step reasoning (GSM8K), in points against stock, for the chat fine-tunes P2 to P5 and for switching the gate inside a reply; the shaded region passes both bars. **c**, Opening the sheet on held-out problems: precision against recall, for stock Qwen with four examples (cobalt) and the fine-tunes (rose).

it by a factor of 1.0065. Yet with that adapter on, the model's instruction following dropped by 16.4 points on IFEval, a benchmark of prompts with checkable instructions[29]. Perplexity does not see the damage.

The gate. We therefore built the adapter into the graph behind a gate, in the spirit of progressive networks, which add capacity beside a frozen model instead of changing it[30]. Every adapter branch's output is multiplied by one input of the graph, g, before it is added back: $y = xW + g\,s\,(xA^{\top})B^{\top}$, where W is the frozen weight, A and B the adapter's matrices and s its fixed scale (Fig. 5a). At $g = 0$ every patched layer adds a floating-point zero, which returns the other operand unchanged, so the model is stock Qwen3.5-0.8B; at $g = 1$ it is the sheet machine. On ONNX Runtime, over 2,500 chat forward passes and 2,688 table views, every logit, hidden state and cached state of the gated graph equals stock at $g = 0$ and the ungated sheet machine at $g = 1$, bit for bit, and greedy replies to all 100 chat prompts are identical. On the M4 the certificate at $g = 1$ reproduced all 2,446 margins. The page's chat runs at $g = 0$, so its language is stock by construction.

Deciding to open the sheet. The goal is a model that, in the middle of a reply, opens its sheet, writes a short work order, lets the certified procedures run, and continues with the results. That decision must live in weights that are on during chat, so the gate cannot provide it. We train it on synthetic dialogues (problems that need the sheet, distractors that do not, and problems inside prose) and score it on phrasings and scenarios held out from training. To keep the model's language, training adds a penalty on the divergence from the gate-0 model's own next-token distributions, on its replies to instruction prompts (self-distillation, as in Learning without Forgetting[31]) and on the model's own samples (on-policy distillation[32,33]), the regime in which fine-tuning is known to forget least[34].

Today. The fine-tune has not yet passed all of its registered bars. The best adapter so far opens the sheet on held-out problems with precision 1.000 and recall 0.991, keeps IFEval at stock (0.538 against 0.536; paired flips 52 won, 51 lost), writes correct work orders for 96% of held-out equations, and keeps all 2,446 certified views and perplexity within 1%. Three bars are still short: correct work orders for 85.6% of held-out word problems against a bar of 90%, and grade-school word problems (GSM8K[35]) and general knowledge (MMLU[36]) 2.4 and 2.2 points below stock against a bar of 2. Until all pass, the page's chat runs at the gate's stock setting.

The runs so far show a trade-off. Each change that restored multi-step reasoning cost instruction following, until on-policy distillation removed most of that cost (Fig. 5b). Opening the sheet is the easy part: stock Qwen, shown four examples, opens it with recall 0.81; the fine-tunes, shown none, with recall 0.946 to 0.998 (Fig. 5c). The hard part is the language around it: translating a word problem into the right expression, which the sheet does not check.

7 Inside the model

Where in the network is a step decided? The logit lens reads the residual stream, the running sum of every layer's output, through the model's final normalisation and output matrix after each layer, as if the network

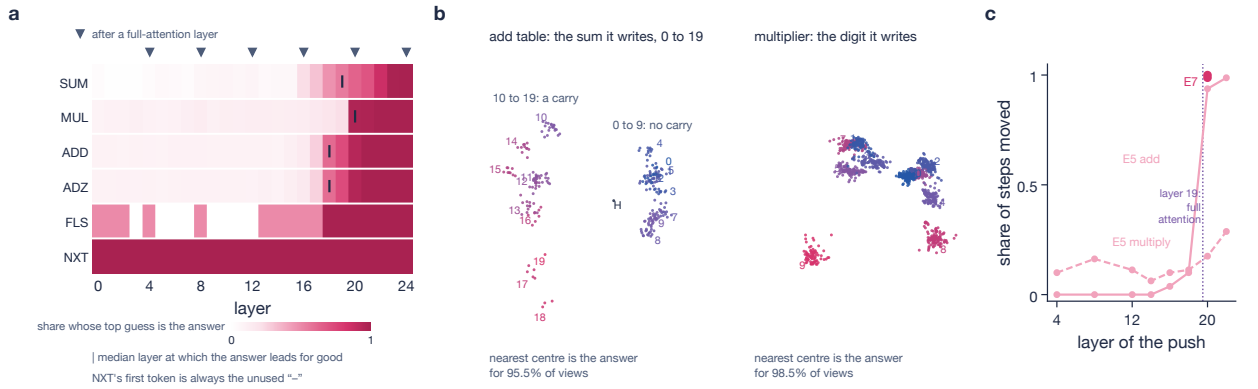


Fig. 6 | Inside the model. a, The logit lens: the share of views whose top token after each depth is the final answer, per step word; ticks mark the median depth at which the answer leads for good, triangles the ends of the full-attention layers. **b**, The states that write the answer, projected to three dimensions at the last layer and coloured by answer: addition (the sum written, 0 to 19) and the multiplier's digit. **c**, A class-mean push at the answer position: the share of steps changed to the pushed answer against the depth of the push, for an earlier adapter (E5, addition and multiplication) and the published one (E7, at depth 20).

stopped there[37,38]. We count depth as the number of layers applied; the six full-attention layers end at depths 4, 8, 12, 16, 20 and 24. For the published model, the answer takes the lead for good at a median depth of 19 for addition, 20 for multiplication and 18 for the add steps of the multiplier, and through depth 15 it is the top token for at most 13% of the views of any of these tables (Fig. 6a).

The decision can be moved. A class-mean push adds to the residual stream, at the answer position, the difference between the mean state of a target answer and that of the true one[39,40]. Applied after depth 20, the end of the fifth full-attention layer, it changes 79 of 80 addition steps and 80 of 80 multiplication steps to the pushed answer. In an earlier adapter the same push moved 8 of 80 addition steps when applied after depth 18 and 75 of 80 after depth 20: the step is decided in or just before the full-attention layer that ends at depth 20 (Fig. 6c). In a three-dimensional projection of the final layer, the states of the views gather by the answer they write, and the sums that carry (10 to 19) lie apart from those that do not (Fig. 6b). The page runs this push on the visitor's own GPU, through an extra input of the graph, and shows the next step change.

8 Related work

Neural arithmetic. Learned exact arithmetic at unbounded length has a long line. The Neural GPU learned binary addition and multiplication with convolutional updates that generalise far beyond training lengths[41], with failures on structured inputs such as long runs of zeros[42]. Arithmetic units with built-in inductive biases followed[43,44]. For transformers, positional encodings and data formats decide whether addition generalises to longer inputs[3–9], and fine-tuned language models reach near-perfect accuracy on bounded arithmetic[45,46], with division and the greatest common divisor the hard cases[47–49]. Neural cellular automata and differentiable transducers trained from scratch now add and multiply exactly at hundreds to thousands of times their training length[50–52]. Mechanistic studies find addition circuits that are correct on all but a vanishing fraction of inputs[53], and exactness has been proved for idealised training[54]. Sketchpad differs in three ways: the executor is a pretrained vision-language model with a small adapter, not a network trained from scratch; its procedures include division, square roots and the greatest common divisor; and exactness is certified by exhausting the views, not estimated on samples.

Procedures inside language models. CAEF fine-tunes Llama-3.1-8B to step through arithmetic operators, division included, on a text tape, with one adapter per operator, reaching nearly 100% up to 100 digits without a certificate[55]. OccamLLM and a gated calculator place exact arithmetic inside a language model as a built-in, non-learned module[56,57]; compiled and looped transformers run programs with hand-set weights[58–61]; and a 60-million-parameter transformer trained as an interpreter predicts the steps of program traces thousands of lines long[62]. Chain of Code lets a language model emulate an interpreter for lines that cannot run[63]. In Sketchpad the procedure is learned, each step is one forward pass of a pretrained model, and nothing is carried between steps but the sheet.

Tools. Toolformer showed that a model can learn when to call a tool by keeping the calls that help[10]; PAL, Program of Thoughts, ToRA, ReTool and ToolRL interleave reasoning with code or tool calls[11–14,64], and recursive language models call themselves through an interpreter[65]. Our chat fine-tune borrows Toolformer's recipe, with the model's own certified computation as the tool.

Verification on finite sets. Neural-network verification usually bounds a network's output over continuous regions of input[66]. Our inputs are finite by construction, so verification is enumeration. Moore showed that experiments on a machine identify it only given a bound on its states[15], the basis of conformance testing[67]. Cai et al. proved learned recursive programs correct by checking a finite verification set[16]. Schuurmans et al. and Lewandowski et al. certified universal computation by a language model's decoding by checking every production rule of a universal machine[17,68], Abbe et al. proposed the same context discipline, keeping the question and the current state and dropping the rest, as the inductive scratchpad[69], and PENCIL erases intermediate reasoning to keep the context short[70]. Certification from examples can be hard under small changes of architecture, and length generalisation bounds can be uncomputable[71,72]; our guarantee needs no such bound, because the pinned-sheet policy makes the set of inputs finite. Our theorems are lifts of these results to a sheet; we claim the instance, its formal statement and its check on the deployed device.

Reading rendered text. Vision encoders can read text as pixels well enough to model language[73–75], and rendered text is now used to compress long contexts[76,77], though vision-language models that read digits reliably still fail at the multiplication[78]. Visual scratch pads let multimodal models draw and reread diagrams as a visual chain of thought[79–83]. Our sheet is narrower and exact: one fixed glyph set, read three cells at a time, where untouched Qwen3.5-0.8B reads single cells exactly but long rendered numbers only to 98 to 99% per digit. The name Sketchpad nods to Sutherland's interactive drawing system[84].

Algebra. Transformers have learned symbolic integration[85], step-by-step polynomial simplification with the arithmetic offloaded to a calculator[86], Gröbner bases[87] and, as large language models, school algebra in chain of thought[88]; theory shows that chain of thought lets small transformers solve arithmetic and linear equations[89]. None is exact by construction. Our algebra algorithms are classical; what is new is that every number in them comes from a certified learned executor.

Continual learning. Learning without Forgetting distils the old model into the new one[31]; progressive networks keep the old weights and add new ones beside them[30]; elastic weight consolidation protects important weights[28]; X-LoRA routes between adapters[90]. On-policy distillation and RL's Razor show that forgetting tracks the divergence from the base on the model's own outputs[32–34]. Our gate is a hard switch of the progressive kind: it guarantees stock behaviour at 0, and leaves the decision to compute to the fine-tune.

What we did not find. In a search of the literature on 30 October 2026, covering the work above and 50 arXiv abstracts read in full (Methods), we found no earlier pretrained vision-language model that does exact arithmetic at any length with a rendered sheet as its only state, certified on every view and checked again on the user's device; no learned, certified long division, square root or greatest common divisor; and no algebra whose every arithmetic step is done by a learned, certified executor.

9 Limits

The certificate covers the model's steps and nothing else. The runtime's layout, its choice of procedure, and the transcription of a problem into a work order are not certified; they are ordinary code and, in chat, ordinary language. The theorems assume exact arithmetic; on hardware they hold through the margins, and the device certificate holds for the device and path that measured it. A visitor's GPU checks the weakest tenth of the shipped table, not all of it, unless asked. The algebra covers polynomial equations of small degree in one unknown and linear systems in up to three, and it checks the roots it finds, not that it found all of them. The model reads typed problems drawn in its own glyphs, not photographs. And it is slow: a step costs about 0.1 s on a laptop GPU, so a thousand-digit sum takes over a minute and some square roots and expansions take more than 12 s. The model that decides for itself when to compute is still in training.

Methods

Model and adapter

We use Qwen3.5-0.8B[18] (language model: 24 layers, hidden size 1,024; layers 4, 8, …, 24 are full attention with 8 query and 2 key-value heads, the rest Gated DeltaNet[20]). The adapter is a LoRA of rank 16 and scale 32/16 on the query, key, value and output projections of the 6 attention layers, the input and output

projections of the 18 Gated DeltaNet layers and the three projections of all 24 feed-forward blocks: 150 matrices and 10,223,616 parameters, against 873,438,784 in the base model with its vision encoder. For the browser the model is exported to ONNX: the token embedding in 4-bit, the vision encoder and the language model in 16-bit floating point; the adapter is folded into a 21 MB decoder graph that shares the base's external weights. Two extra graph inputs serve the page: the gate (`lora_gate`, Section 6) and a push vector added to the residual stream after depth 20 (Section 7); one extra output returns the hidden states for the inside view.

Views, prompts and tables

Glyphs are 32×32 pixel sprites of blank and the ten digits, rendered once and reused for every view, so every view the model can see is one of a finite set of exact images. A view stacks three cells into a 32×96 image, which the processor, set to its native size, turns into three image tokens. The prompt of a step is its word, the image, `=` and two `?` slots, padded with further `?` to the length of the longest word; the answers are read at `=` and the two slots. Extended Data Table 1 lists the nine tables. The rows are exhaustive over each step's possible views: for example, `MUL` covers every digit or blank of the multiplicand, every digit of the multiplier and every carry from blank to 8 ($11 \times 10 \times 10 = 1{,}100$), and `EST` covers every remainder pair under every normalised divisor digit from 5 to 9 (400). That the procedures meet no other view follows from the layout and, for division and roots, from normalisation; it is also checked by the runtime on every step, and by the reference runtime on 1,000 random cases per procedure, with no view outside the tables.

Training

Training reads the tables, not sampled problems: every step draws rows from each table, and the loss is cross-entropy plus a hinge that pushes every margin above 4, plus a penalty on the divergence from the base model's next-token distributions on plain text[31]. Every 100 steps the whole certificate is computed; rows with margins below 2 are replayed, and training stops when every row passes with margin at least 3. The published adapter, E7, continued from the adapter E6, which already passed the addition and multiplication tables, with the subtraction, guess and lowering tables added. It passed every row first at step 1,200 and stopped at step 1,600, after 19.8 minutes on one H100 GPU (about $1.30), with the learning rate decaying on a cosine from 2×10^{-4} to 2×10^{-5}. Its plain-text perplexity is 1.0048 times stock. End to end on the server it multiplied 20 of 20 four-digit, 10 of 10 six-digit and 5 of 5 ten-digit pairs and added 20 of 20 ten-digit, 5 of 5 hundred-digit and 2 of 2 thousand-digit pairs exactly.

Certification

A margin is the correct token's logit minus the largest logit of any other token in the vocabulary of 248,320, at each answer position; a row passes when every answer position has a positive margin. The server certificate runs in PyTorch in 32-bit floating point. The device certificate runs the deployed files (transformers.js 4.2.0[22], ONNX Runtime Web on WebGPU) on an Apple M4, through the page's own code path and padded batch form, and records every margin together with the device, the library version, the hash of every file and the sprite sheet. The page accepts a shipped certificate only for identical files, and checks the weakest tenth of each table again on the visitor's GPU before a run. The certificate file format is described in Appendix B.

Procedures and timing

Signed sums and differences run `SUM` or `SUB` column by column; a subtraction ends on a halt row that reports whether a borrow came out, in which case the subtraction runs the other way round and the sign flips. Long division follows Algorithm D[24]: both numbers are scaled by $f = \lfloor 10/(d+1) \rfloor$, d the divisor's leading digit, in one multiplication pass each; each quotient digit is guessed by `EST` from the remainder's top two digits and the divisor's top digit (or skipped when the guess is zero); a multiply-subtract pass runs `MUL` one column ahead of `SUB`, two views per call; a borrow out of the top column triggers `DEC` and an add-back with `SUM`. Square roots scale the radicand by c^2 with $c \in \{1, 2, 4, 5\}$ so the root starts with 25 to 49, which we verified for every radicand from 1 to 999,999; the first guess is 4, the second 9 when the root starts with 2, and every later guess comes from `EST`. The runtime picks f, c and the first two guesses from the leading digits it lays out; each guess is at least the true digit and is lowered only by a certified multiply-subtract and `DEC`, so these choices can cost steps but cannot change a digit. Greatest common divisors run Euclid's algorithm by

repeated division. The cost model, fitted on batches of 1 to 32 rows on the M4 with layer states left on the GPU, is 0.065 s per forward pass plus 0.037 s per view.

The registered browser test (Extended Data Table 2) drew its 200 problems from a fixed generator (seed 12345) before any run: $a \div b$ to 10 places with $a \le 999$ and $2 \le b \le 99$; $\sqrt{a}$ to 6 places with $2 \le a \le 999$; $\gcd(a, b)$ with $10 \le a, b \le 999$; and signed sums and differences of 1 to 20 digits. It ran in Chrome on an Apple M4 in the page's fast mode, one problem after another, with wall time measured from the call to the answer. The WebGPU device was lost three times during the run; each time the model was reloaded and the interrupted problem redone, and only completed runs are counted. Every answer was checked against the page's own big-integer arithmetic and again in Python. The run was repeated in full with the gated published model: all 200 answers were again exact, and every step's views and answers were identical.

Algebra

The algebra runtime (`program/`) parses a problem, writes the derivation line by line, and obtains every number through the calculator's interface, which runs certified procedures on the sheet; independent calls share waves. Each line records which model calls produced its numbers, which the page uses to light each number when its call completes (Fig. 4a). The light check confirms collected coefficients at one further point and substitutes rational roots into the collected equation; the strong check substitutes into the typed sides. Irrational roots of a quadratic $ax^2 + bx + c$ are written as a reduced pair $(p \pm q\sqrt{m})/d$ and checked by the integer identities $a \cdot 2p = -b\,d$ and $a(p^2 - q^2 m) = c\,d^2$, which hold exactly when the pair is the quadratic's two roots; square factors are extracted by trial of the primes up to 50. The fault populations planted a wrong root (200 per route) or corrupted the result of one call inside the initial evaluation or collection work (200 per route, fixed seeds); a second population perturbed one parameter of 200 radical pairs, and all 200 were refused in both check modes. The reference test suite of 264 tests, including 2,200 generated problems per route and check mode, runs with exact reference tables in place of the model; every cost is computed by the cost model from rows and calls, not measured on the device.

The gate

The gate multiplies each LoRA branch's output before it is added to the frozen projection, in all 150 patched matrices. The identity test ran on ONNX Runtime 1.31.1 on CPU, with three graphs on the same external weights: stock (the base with the inside patch), the ungated adapter and the gated one. It compared every logit, hidden state and cached state, bit for bit, on 100 chat prompts (the first user turn of the No Robots test split[91]; one prefill and 24 greedy decoding steps each, 2,500 calls) and on all 2,446 table views plus the 242 addition views in padded form, with inputs captured from the reference forward pass.

The chat fine-tune

The training set has about 3,150 synthetic dialogues, of which about 30% are distractors that must not open the sheet, plus 547 grade-school training problems solved by stock Qwen in its own words, with each calculation turned into a sheet call and kept only when the stand-in sheet reproduced it. The held-out set has 780 items from templates never trained on. Every work order is scored by running it, never by string match. Registered bars, set before any run: precision and recall of opening the sheet at least 0.95; correct work orders for at least 90% of word problems and 95% of equations and expressions; IFEval strict prompt-level accuracy[29] within 2 points of stock; and, from the second round, GSM8K (250 test problems) and MMLU (500 questions) within 2 points of stock. Kill lines: any certified view lost, perplexity above 1.03 times stock, or IFEval more than 5 points down. Paired flips (the prompts only one of two models gets right) give the noise of each comparison. Extended Data Table 4 lists every run.

Inside the model

The logit lens applies the final normalisation and output matrix to the residual stream after each depth at the first answer position, for the 1,795 addition and multiplication views; a view's decision depth is the first depth after which the final answer stays the top token. The class-mean push replaces the answer position's state after depth 20 by itself plus the mean state of the target answer's views minus the mean of the true answer's, and counts how often the step's answer becomes the target (80 trials per table).

Extended Data Table 1 | The nine tables. Each row is one view the step can meet. Margins are the smallest over the table, in logits, for the published model: on the server in 32-bit PyTorch, and on the M4 in the browser in 16-bit, from the shipped certificate. Next-step letters: `A` add the partial product, `Z` end of pass, `M` multiply (or multiply-subtract), `F` flush, `N` next digit, `H` halt, `B` borrow out, `S` subtract on, `C` correct the remainder.

Step	View (top to bottom)	Answers at `=`, `?`, `?`	Rows	Server	M4
`SUM`	carry (blank, 1), top digit, bottom digit (blank, 0–9)	sum and carry as one token (0–19), or `H`	242	3.96	3.02
`MUL`	multiplicand digit (blank, 0–9), multiplier digit (0–9), carry (blank, 0–8)	digit, carry, `A` or `Z`	1,100	6.39	6.52
`ADD`	carry (blank, 1), product digit, running-total digit (blank, 0–9)	digit, carry, `M`	220	9.37	9.50
`ADZ`	as `ADD`, at the end of a pass	digit, carry, `F`	220	8.41	8.00
`FLS`	carry (blank, 1), blank, blank	the last carry, –, `N`	2	12.93	13.11
`NXT`	next multiplier digit (blank, 0–9), blank, blank	–, –, `M` or `H`	11	10.48	10.22
`SUB`	borrow (blank, 1), top digit, bottom digit (blank, 0–9)	digit, borrow, `S`; all blank: –, –, `H` or `B`	242	8.36	8.19
`EST`	remainder's top two digits $r_1 \leq d$, divisor's top digit d (5–9)	$\min(9, \lfloor(10r_1 + r_2)/d\rfloor)$, –, `M` or `N`	400	4.51	4.61
`DEC`	guess q (1–9), blank, blank	$q - 1$, –, `C`	9	10.00	10.09
All			**2,446**	**3.96**	**3.02**

Baselines

The baseline of Fig. 1c was registered before it ran (`paper/BASELINES.md`): 220 problems, 20 per length, sums of 10, 30, 100, 300 and 1,000 digits and products of 2-, 4-, 6-, 10-, 15- and 20-digit numbers (seed 20261031), the same prompt for every model ("What is a + b? Give the exact integer. Put the final answer in \boxed{}."), Qwen's recommended sampling for each mode, and a budget of 32,768 new tokens with chain of thought and 4,096 without. The stock models ran under vLLM on one H100 GPU. The 27-billion-parameter slot used the FP8 release of Qwen3.5-27B, because the 16-bit model held only 8 replies at full length and would have exceeded the budget; the choice was fixed before scoring. Sketchpad ran in PyTorch with the published adapter, all 220 problems in lockstep. The answer was the last boxed integer after the reasoning, separators removed; a reply that hit the budget or gave no answer counted as wrong. The run cost about $7 on Modal.

Literature search

The prior-art search of 30 October 2026 read 50 arXiv abstracts and the proof method of Cai et al., on exact arithmetic by learned models, certification of learned programs, procedure-stepping and compiled arithmetic in language models, algebra by learned models, tool use and adapter switching. It is a search, not a proof of absence. Every reference in this paper was checked against its source on 31 October 2026 (`paper/refs_verified.md`).

Data and code availability

The model, its adapter, the certificate and the inside data are on Hugging Face (`mohsenvand/sketchpad-0.8b-preview`, revision `2df87172`). The runtime, the page, the algebra, the baselines and this paper's sources are in the Sketchpad repository under the Apache 2.0 licence. The training and evaluation scripts and the records cited in `paper/numbers.md` are in the polyformer repository (`research/sketch/`) and in this one. The Lean proofs are in the polyformer package `research/lean` (branch `lean-p`) and in `paper/lean/Sheet.lean`.

Extended Data Table 2 | The registered browser test. 50 problems per set on the Apple M4 (WebGPU, 16-bit decoder), ungated E7; the full certificate passed first.

Set	Exact	Median s	Max s	Over 12 s	Median calls / views	Cost model median s
$a \div b$ to 10 places ($a \leq 999$, $b \leq 99$)	50	7.0	13.3	1	65 / 90	7.6
$\sqrt{a}$ to 6 places ($a \leq 999$)	50	9.8	16.5	11	86 / 137	10.7
$\gcd(a, b)$ ($a, b \leq 999$)	50	7.0	13.4	4	61 / 84	7.1
signed $\pm$, up to 20 digits	50	2.0	4.7	0	19 / 19	1.9

Extended Data Table 3 | Cost of the worked algebra examples. Rows (views), forward passes after waves and seconds by the cost model, light check, for the textbook and evaluation routes.

Work order	Textbook rows / passes / s	Evaluation rows / passes / s
`solve 3(x-2)+5=2x+14`	36 / 22 / 2.8	75 / 27 / 4.5
`solve 7x+13=2026-4x`	97 / 62 / 7.6	165 / 88 / 11.8
`solve 2x+3y=7, x-y=1`	75 / 48 / 5.9	194 / 41 / 9.8
`solve (2x-3)(x+5)=x^2+1-16`	79 / 43 / 5.7	177 / 50 / 9.8
`solve x^2-x-6=0`	102 / 77 / 8.8	162 / 79 / 11.1
`123456 * 654321`	96 / 46 / 6.5	96 / 46 / 6.5
`expand (x+2)^5`	159 / 95 / 12.1	458 / 248 / 33.1
`expand (2x-3)^4`	172 / 94 / 12.5	492 / 274 / 36.0
`factor x^3-6x^2+11x-6`	195 / 149 / 16.9	483 / 210 / 31.5
`expand (x+1)^10`	322 / 154 / 21.9	2,359 / 845 / 142.2
`solve 2x+3y-z=1, x-y+2z=3, 3x+y+z=8`	323 / 167 / 22.8	538 / 102 / 26.5
`solve x^2-2=0` (exact answer)	86 / 65 / 7.4	
`solve 2x^2+3x-1=0` (exact answer)	144 / 88 / 11.0	
`solve x^2-6x+4=0` (exact answer)	133 / 96 / 11.2	

Extended Data Table 4 | The chat fine-tunes against their registered bars. Held-out templates at gate 1 with no examples, against stock Qwen with four examples (opening and work orders) or stock with none (benchmarks). IFEval, GSM8K and MMLU are in points against stock (0.536, 0.516 and 0.476). Bars as registered; GSM8K and MMLU became bars in the second round. P7 is running.

Run	Precision / recall	Word	Equations	Prose	IFEval	GSM8K	MMLU	Views kept
bar	≥ 0.95 / ≥ 0.95	≥ 0.90	≥ 0.95	≥ 0.95	≥ -2	≥ -2	≥ -2	all
stock, four examples	0.971 / 0.809	0.789	0.583	0.667	0	0	0	
P1 (from E6)	0.957 / 0.996	0.872	0.967	0.983	−4.8		−6.6	1,795
P2	1.000 / 0.998	0.894	0.975	1.000	−0.55	−31.2	−5.4	2,446
P3	1.000 / 0.994	0.856	0.975	0.992	−2.8	−14.0	−2.4	2,446
P4	1.000 / 0.994	0.817	0.958	0.992	−4.4	−6.4	−3.4	2,446
P5 (on-policy)	1.000 / 0.946	0.678	0.958	0.983	−1.5	−2.0	−2.2	2,446
P6 (wider word data)	1.000 / 0.991	0.856	0.958	0.975	+0.2	−2.4	−2.2	2,446

Extended Data Table 5 | The registered baseline. Problems answered exactly, of 20 per cell, with the number of replies that reached the token budget in brackets. Qwen3.5-27B is the FP8 release.

System	+10	+30	+100	+300	+1,000	×2	×4	×6	×10	×15	×20	All
Sketchpad	20	20	20	20	20	20	20	20	20	20	20	220
Qwen3.5-27B, thinking	18 (2)	12 (8)	8 (10)	1 (19)	0 (20)	20 (0)	20 (0)	15 (4)	14 (6)	6 (11)	7 (13)	121
Qwen3.5-0.8B, thinking	0 (20)	0 (20)	0 (20)	0 (20)	0 (20)	3 (17)	0 (20)	0 (20)	0 (20)	0 (20)	0 (20)	3
Qwen3.5-0.8B, no thinking	0 (8)	0 (20)	0 (20)	0 (20)	0 (20)	15 (4)	8 (9)	0 (14)	0 (20)	0 (19)	0 (20)	23

Extended Data Table 6 | Lean statements. Files in the polyformer package `research/lean/Polyformer/` (branch `lean-p`, commit `a114d08b`), except `Sheet.lean`, which is `paper/lean/Sheet.lean` in this repository and builds against the package.

Theorem	Lean name	File	Axioms
1, sheet form	`Sketchpad.sheet_exact_iff`, `Sketchpad.sheet_certified`	`Sheet.lean`	`propext`, `Quot.sound`
1, token form	`pinned_regeneration_iff`, `regeneration_trace_iff`, `pinned_regeneration_decoder`	`Pinned.lean`	standard
2	`no_finite_test_suite`, `no_finite_check`	`Unpinned.lean`	standard
3	`compose_exact`, `error_decomposition_certified`, `error_bound`	`Compose.lean`	standard
4	`sel_eq_of_close`, `regeneration_of_close`	`Pinned.lean`	standard
4, the cache	`policy_tick_eq`, `policy_regeneration`	`Policy.lean`	`propext`, `Quot.sound`

Extended Data

The Lean package

The theorems are proved in Lean 4[92] with Mathlib[93] (Lean and Mathlib 4.35.0). The package builds with `lake build` (3,150 jobs) and contains no `sorry`, `admit` or added axiom; `#print axioms` on every theorem below lists only `propext`, `Classical.choice` and `Quot.sound`, Lean's standard axioms, or a subset. The model in Lean is an abstraction: the network is a deterministic function of its context (or a causal decoder whose layer at a position reads only earlier positions), the argmax may break ties any way, and arithmetic is exact; floating point enters only through Theorem 4's τ.

The certificate format

A certificate is one JSON file shipped beside the model graph (`arithmetic/certificate.json`). Its `format` string names the prompt forms it was measured in (for the published model, the padded addition prompt and the slotted, padded multiplier and calculator prompts, on 32×96 strips from the sprite sheet). It records the numerical precision of each graph, the transformers.js version, a fingerprint of the sprite sheet and of every model file, the device's GPU description and the time. For each table it lists the number of rows, the number that passed and every row's margin, in the table's fixed row order, to a thousandth of a logit. The page accepts it only if every fingerprint matches the files it loaded, and then checks the rows with the smallest tenth of margins again on the visitor's GPU.

Reproducing the numbers

Every number in this paper is listed in `paper/numbers.md` with the record it comes from. Training, certification on the server, the identity test of the gate and the chat runs are Modal jobs in polyformer `research/sketch/` (`e7_calc.py`, `e7_gate_identity.py`, `chat/`), each logged with its job id and cost. The device certificate and the registered browser test run on the page itself (`sketchpad.exportCertificate()` and `sketchpad.runCalc()` in the browser console). The algebra's tests and costs run with `pnpm vitest run src/lib/sketchpad/program`, and the costs are checked independently by `verify_costs.py` and `verify_round3.py`. The baseline is `paper/baselines/` (`spawn.py` launches it, `score.py` rebuilds `results.json` from the raw replies). The Lean proofs build as described above.

References

1. Wei, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Advances in Neural Information Processing Systems, 24824–24837 (2022). https://arxiv.org/abs/2201.11903
2. Nye, M. et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models. Preprint at arXiv:2112.00114 (2021). https://arxiv.org/abs/2112.00114
3. Nogueira, R., Jiang, Z. & Lin, J. Investigating the Limitations of Transformers with Simple Arithmetic Tasks. Preprint at arXiv:2102.13019 (2021). https://arxiv.org/abs/2102.13019
4. Anil, C. et al. Exploring Length Generalization in Large Language Models. In Advances in Neural Information Processing Systems, 38546–38556 (2022). https://arxiv.org/abs/2207.04901
5. Lee, N., Sreenivasan, K., Lee, J. D., Lee, K. & Papailiopoulos, D. Teaching Arithmetic to Small Transformers. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2307.03381
6. Zhou, H. et al. What Algorithms can Transformers Learn? A Study in Length Generalization. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2310.16028
7. Zhou, Y. et al. Transformers Can Achieve Length Generalization But Not Robustly. Preprint at arXiv:2402.09371 (2024). https://arxiv.org/abs/2402.09371
8. McLeish, S. et al. Transformers Can Do Arithmetic with the Right Embeddings. In Advances in Neural Information Processing Systems, 108012–108041 (2024). https://doi.org/10.52202/079017-3430
9. Cho, H. et al. Position Coupling: Improving Length Generalization of Arithmetic Transformers Using Task Structure. In Advances in Neural Information Processing Systems, 22233–22315 (2024). https://doi.org/10.52202/079017-0701
10. Schick, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In Advances in Neural Information Processing Systems, 68539–68551 (2023). https://doi.org/10.52202/075280-2997
11. Gao, L. et al. PAL: Program-aided Language Models. In Proceedings of the 40th International Conference on Machine Learning, 10764–10799 (2023). https://arxiv.org/abs/2211.10435
12. Chen, W., Ma, X., Wang, X. & Cohen, W. W. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. Transactions on Machine Learning Research (2023). https://arxiv.org/abs/2211.12588
13. Gou, Z. et al. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2309.17452
14. Feng, J. et al. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. In International Conference on Learning Representations (ICLR 2026) (2026). https://arxiv.org/abs/2504.11536
15. Moore, E. F. Gedanken-Experiments on Sequential Machines. In Automata Studies, 129–153 (1956). https://doi.org/10.1515/9781400882618-006
16. Cai, J., Shin, R. & Song, D. Making Neural Programming Architectures Generalize via Recursion. In International Conference on Learning Representations (ICLR 2017) (2017). https://arxiv.org/abs/1704.06611
17. Lewandowski, A., Machado, M. C. & Schuurmans, D. Universal computation is intrinsic to language model decoding. Preprint at arXiv:2601.08061 (2026). https://arxiv.org/abs/2601.08061
18. Qwen Team. Qwen3.5: Towards Native Multimodal Agents. Qwen blog (2026). https://qwen.ai/blog?id=qwen3.5
19. Hu, E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations (ICLR 2022) (2022). https://arxiv.org/abs/2106.09685
20. Yang, S., Kautz, J. & Hatamizadeh, A. Gated Delta Networks: Improving Mamba2 with Delta Rule. In International Conference on Learning Representations (ICLR 2025) (2025). https://arxiv.org/abs/2412.06464
21. ONNX Runtime developers. ONNX Runtime. Software. Source https://github.com/microsoft/onnxruntime, MIT license; citation per the repository CITATION.cff (date-released 29 November 2018); the demo uses onnxruntime-web 1.26.0-dev.20260416 through Transformers.js 4.2.0 (2018). https://onnxruntime.ai
22. Hugging Face. Transformers.js. GitHub repository. Software, version 4.2.0 (released 23 April 2026), Apache-2.0 (2026). https://github.com/huggingface/transformers.js
23. Ninomiya, K., Jones, B. & Blandy, J. WebGPU. W3C Candidate Recommendation Draft, 15 September 2026. Latest version: https://www.w3.org/TR/webgpu/ (2026). https://www.w3.org/TR/2026/CRD-webgpu-20260915/
24. Knuth, D. E. The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Third edn, Addison-Wesley (1997).
25. Kaltofen, E. & Trager, B. M. Computing with Polynomials Given by Black Boxes for Their Evaluations: Greatest Common Divisors, Factorization, Separation of Numerators and Denominators. Journal of Symbolic Computation **9**, 301–320 (1990). https://doi.org/10.1016/S0747-7171(08)80015-6
26. Kronecker, L. Grundzüge einer arithmetischen Theorie der algebraischen Grössen. Journal für die reine und angewandte Mathematik **92**, 1–122 (1882). https://doi.org/10.1515/crll.1882.92.1
27. McCloskey, M. & Cohen, N. J. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In Psychology of Learning and Motivation, 109–165 (1989). https://doi.org/10.1016/S0079-7421(08)60536-8
28. Kirkpatrick, J. et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy

of Sciences **114**, 3521–3526 (2017). https://doi.org/10.1073/pnas.1611835114
29. Zhou, J. et al. Instruction-Following Evaluation for Large Language Models. Preprint at arXiv:2311.07911 (2023). https://arxiv.org/abs/2311.07911
30. Rusu, A. A. et al. Progressive Neural Networks. Preprint at arXiv:1606.04671 (2016). https://arxiv.org/abs/1606.04671
31. Li, Z. & Hoiem, D. Learning without Forgetting. IEEE Transactions on Pattern Analysis and Machine Intelligence **40**, 2935–2947 (2018). https://doi.org/10.1109/TPAMI.2017.2773081
32. Agarwal, R. et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2306.13649
33. Lu, K. & Thinking Machines Lab. On-Policy Distillation. Thinking Machines Lab: Connectionism. Blog post, 27 October 2025 (2025). https://doi.org/10.64434/tml.20251026
34. Shenfeld, I., Pari, J. & Agrawal, P. RL's Razor: Why Online Reinforcement Learning Forgets Less. In International Conference on Learning Representations (ICLR 2026) (2026). https://arxiv.org/abs/2509.04259
35. Cobbe, K. et al. Training Verifiers to Solve Math Word Problems. Preprint at arXiv:2110.14168 (2021). https://arxiv.org/abs/2110.14168
36. Hendrycks, D. et al. Measuring Massive Multitask Language Understanding. In International Conference on Learning Representations (ICLR 2021) (2021). https://arxiv.org/abs/2009.03300
37. nostalgebraist. interpreting GPT: the logit lens. LessWrong. Blog post, 31 August 2020 (2020). https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens
38. Belrose, N. et al. Eliciting Latent Predictions from Transformers with the Tuned Lens. Preprint at arXiv:2303.08112 (2023). https://arxiv.org/abs/2303.08112
39. Turner, A. M. et al. Steering Language Models With Activation Engineering. Preprint at arXiv:2308.10248 (2023). https://arxiv.org/abs/2308.10248
40. Rimsky, N. et al. Steering Llama 2 via Contrastive Activation Addition. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 15504–15522 (2024). https://doi.org/10.18653/v1/2024.acl-long.828
41. Kaiser, Ł. & Sutskever, I. Neural GPUs Learn Algorithms. In International Conference on Learning Representations (ICLR 2016) (2016). https://arxiv.org/abs/1511.08228
42. Price, E., Zaremba, W. & Sutskever, I. Extensions and Limitations of the Neural GPU. Preprint at arXiv:1611.00736 (2016). https://arxiv.org/abs/1611.00736
43. Trask, A. et al. Neural Arithmetic Logic Units. In Advances in Neural Information Processing Systems (2018). https://arxiv.org/abs/1808.00508
44. Madsen, A. & Johansen, A. R. Neural Arithmetic Units. In International Conference on Learning Representations (ICLR 2020) (2020). https://arxiv.org/abs/2001.05016
45. Liu, T. & Low, B. K. H. Goat: Fine-tuned LLaMA Outperforms GPT-4 on Arithmetic Tasks. Preprint at arXiv:2305.14201 (2023). https://arxiv.org/abs/2305.14201
46. Yang, Z. et al. GPT Can Solve Mathematical Problems Without a Calculator. Preprint at arXiv:2309.03241 (2023). https://arxiv.org/abs/2309.03241
47. Shen, S., Shen, P. & Zhu, D. RevOrder: A Novel Method for Enhanced Arithmetic in Language Models. Preprint at arXiv:2402.03822 (2024). https://arxiv.org/abs/2402.03822
48. Charton, F. Learning the greatest common divisor: explaining transformer predictions. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2308.15594
49. Kasliwal, S. Algorithmic Scratchpads and Curriculum Staging for Arithmetic Reasoning in Tiny Transformers. Preprint at arXiv:2610.09003 (2026). https://arxiv.org/abs/2610.09003
50. Wei, Z. On the Mirage of Long-Range Dependency, with an Application to Integer Multiplication. Preprint at arXiv:2603.29069 (2026). https://arxiv.org/abs/2603.29069
51. Wei, Z. On the Spatiotemporal Dynamics of Generalization in Neural Networks. Preprint at arXiv:2602.01651 (2026). https://arxiv.org/abs/2602.01651
52. Papazov, H., D'Angelo, F. & Flammarion, N. Exact Learning of Arithmetic with Differentiable Agents. Preprint at arXiv:2511.22751 (2025). https://arxiv.org/abs/2511.22751
53. Quirke, P., Neo, C. & Barez, F. Understanding Addition and Subtraction in Transformers. Preprint at arXiv:2402.02619 (2024). https://arxiv.org/abs/2402.02619
54. Back de Luca, A., Giapitzakis, G. & Fountoulakis, K. Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks. In Advances in Neural Information Processing Systems, 87469–87513 (2025). https://doi.org/10.52202/085713-2636
55. Lai, J. et al. Executing Arithmetic: Fine-Tuning Large Language Models as Turing Machines. Preprint at arXiv:2410.07896 (2024). https://arxiv.org/abs/2410.07896
56. Dugan, O. et al. OccamLLM: Fast and Exact Language Model Arithmetic in a Single Step. In Advances in Neural Information Processing Systems, 35665–35699 (2024). https://doi.org/10.52202/079017-1125
57. Dietz, F. & Klakow, D. IGC: Integrating a Gated Calculator into an LLM to Solve Arithmetic Tasks Reliably and Efficiently. Preprint at arXiv:2501.00684 (2025). https://arxiv.org/abs/2501.00684
58. Giannou, A. et al. Looped Transformers as Programmable Computers. In Proceedings of the 40th International

Conference on Machine Learning, 11398–11442 (2023). https://arxiv.org/abs/2301.13196
59. Turkcan, M. K. Loom: A Scalable Analytical Neural Computer Architecture. Preprint at arXiv:2604.08816 (2026). https://arxiv.org/abs/2604.08816
60. Percepta. transformer-vm. GitHub repository. Software (Apache-2.0): compile programs directly into transformer weights; repository created 25 March 2026; accessed 31 October 2026 (2026). https://github.com/Percepta-Core/transformer-vm
61. Weng, Y. et al. Mastering Symbolic Operations: Augmenting Language Models with Compiled Neural Networks. In International Conference on Learning Representations (ICLR 2024) (2024). https://arxiv.org/abs/2304.01665
62. Xu, R., Yang, C., Li, Y. & McAllester, D. Towards Training Transformers as a Universal Computer. In Findings of the Association for Computational Linguistics: EMNLP 2026, 48933–48954 (2026). https://doi.org/10.18653/2026.findings-emnlp.2284
63. Li, C. et al. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. In Proceedings of the 41st International Conference on Machine Learning, 28259–28277 (2024). https://arxiv.org/abs/2312.04474
64. Qian, C. et al. ToolRL: Reward is All Tool Learning Needs. In Advances in Neural Information Processing Systems, 116896–116926 (2025). https://doi.org/10.52202/085713-3524
65. Zhang, A. L., Kraska, T. & Khattab, O. Recursive Language Models. Preprint at arXiv:2512.24601 (2025). https://arxiv.org/abs/2512.24601
66. Katz, G., Barrett, C., Dill, D. L., Julian, K. & Kochenderfer, M. J. Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks. In Computer Aided Verification (CAV 2017), 97–117 (2017). https://doi.org/10.1007/978-3-319-63387-9_5
67. Chow, T. S. Testing Software Design Modeled by Finite-State Machines. IEEE Transactions on Software Engineering **SE-4**, 178–187 (1978). https://doi.org/10.1109/TSE.1978.231496
68. Schuurmans, D., Dai, H. & Zanini, F. Autoregressive Large Language Models are Computationally Universal. Preprint at arXiv:2410.03170 (2024). https://arxiv.org/abs/2410.03170
69. Abbe, E., Bengio, S., Lotfi, A., Sandon, C. & Saremi, O. How Far Can Transformers Reason? The Globality Barrier and Inductive Scratchpad. In Advances in Neural Information Processing Systems, 27850–27895 (2024). https://doi.org/10.52202/079017-0874
70. Yang, C., Srebro, N., McAllester, D. & Li, Z. PENCIL: Long Thoughts with Short Memory. In Proceedings of the 42nd International Conference on Machine Learning, 71092–71156 (2025). https://arxiv.org/abs/2503.14337
71. Back de Luca, A. & Fountoulakis, K. Certification from Examples is Hard for Circuits and Transformers under Minimal Overparametrization. Preprint at arXiv:2605.22964 (2026). https://arxiv.org/abs/2605.22964
72. Yang, A., Bergsträßer, P., Zetzsche, G., Chiang, D. & Lin, A. W. Length Generalization Bounds for Transformers. In Proceedings of the 43rd International Conference on Machine Learning, 144227–144248 (2026). https://arxiv.org/abs/2603.02238
73. Rust, P. et al. Language Modelling with Pixels. In International Conference on Learning Representations (ICLR 2023) (2023). https://arxiv.org/abs/2207.06991
74. Tschannen, M., Mustafa, B. & Houlsby, N. CLIPPO: Image-and-Language Understanding from Pixels Only. In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 11006–11017 (2023). https://doi.org/10.1109/CVPR52729.2023.01059
75. Lee, K. et al. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In Proceedings of the 40th International Conference on Machine Learning, 18893–18912 (2023). https://arxiv.org/abs/2210.03347
76. Wei, H., Sun, Y. & Li, Y. DeepSeek-OCR: Contexts Optical Compression. Preprint at arXiv:2510.18234 (2025). https://arxiv.org/abs/2510.18234
77. Cheng, J. et al. Glyph: Scaling Context Windows via Visual-Text Compression. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 37145–37158 (2026). https://doi.org/10.18653/v1/2026.acl-long.1722
78. Balter, S. G., Jerzak, E. & Jerzak, C. T. Multiplication in Multimodal LLMs: Computation with Text, Image, and Audio Inputs. In Findings of the Association for Computational Linguistics: ACL 2026, 40766–40780 (2026). https://doi.org/10.18653/v1/2026.findings-acl.2025
79. Hu, Y. et al. Visual Sketchpad: Sketching as a Visual Chain of Thought for Multimodal Language Models. In Advances in Neural Information Processing Systems, 139348–139379 (2024). https://doi.org/10.52202/079017-4423
80. Menon, S., Zemel, R. & Vondrick, C. Whiteboard-of-Thought: Thinking Step-by-Step Across Modalities. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 20016–20031 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1117
81. Li, C. et al. Imagine While Reasoning in Space: Multimodal Visualization-of-Thought. In Proceedings of the 42nd International Conference on Machine Learning, 36340–36364 (2025). https://arxiv.org/abs/2501.07542
82. Akter, S. N., Madaan, A., Lee, S., Yang, Y. & Nyberg, E. Self-Imagine: Effective Unimodal Reasoning with Multimodal Models using Self-Imagination. Preprint at arXiv:2401.08025 (2024). https://arxiv.org/abs/2401.08025
83. Shi, W. et al. MathCanvas: Intrinsic Visual Chain-of-Thought for Multimodal Mathematical Reasoning. In Pro-

ceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 27933–27954 (2026). https://doi.org/10.18653/v1/2026.acl-long.1287

84. Sutherland, I. E. Sketchpad: A Man-Machine Graphical Communication System. PhD thesis, Massachusetts Institute of Technology (1963). http://hdl.handle.net/1721.1/14979
85. Lample, G. & Charton, F. Deep Learning for Symbolic Mathematics. In International Conference on Learning Representations (ICLR 2020) (2020). https://arxiv.org/abs/1912.01412
86. Agarwal, V., Aditya, S. & Goyal, N. Analyzing the Nuances of Transformers' Polynomial Simplification Abilities. Preprint at arXiv:2104.14095 (2021). https://arxiv.org/abs/2104.14095
87. Kera, H., Ishihara, Y., Kambe, Y., Vaccon, T. & Yokoyama, K. Learning to compute Gröbner bases. In Advances in Neural Information Processing Systems, 33141–33187 (2024). https://doi.org/10.52202/079017-1044
88. Lewkowycz, A. et al. Solving Quantitative Reasoning Problems with Language Models. In Advances in Neural Information Processing Systems, 3843–3857 (2022). https://doi.org/10.52202/068431-0278
89. Feng, G. et al. Towards Revealing the Mystery behind Chain of Thought: A Theoretical Perspective. In Advances in Neural Information Processing Systems, 70757–70798 (2023). https://doi.org/10.52202/075280-3100
90. Buehler, E. L. & Buehler, M. J. X-LoRA: Mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. APL Machine Learning **2** (2024). https://doi.org/10.1063/5.0203126
91. Rajani, N. et al. No Robots. Hugging Face dataset. Dataset HuggingFaceH4/no_robots, CC BY-NC 4.0 (2023). https://huggingface.co/datasets/HuggingFaceH4/no_robots
92. de Moura, L. & Ullrich, S. The Lean 4 Theorem Prover and Programming Language. In Automated Deduction – CADE 28, 625–635 (2021). https://doi.org/10.1007/978-3-030-79876-5_37
93. The mathlib Community. The Lean Mathematical Library. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381 (2020). https://doi.org/10.1145/3372885.3373824